

AR Protein Dynamics Developer's Guide

Wake Tech Community College

CSC 492 Team 8

Austin Daugherty
Claire Davis
Subhashni Kadhiresan
Amanda Richardson

North Carolina State University
Department of Computer Science

December 9th 2022

Table of Contents

Table of Contents

1

Installation

3

Installing Project for Development

3

Upgrading Node Package Manager (NPM) Dependencies

3

Backend

4

Expanding the Application Programming Interface (API)

4

Database

5

Frontend

6

Frontend Folder Structure

6

AR Functionality

7

Testing

9

Setting up Environment for Testing

9

Adding to Junit Testing

9

Security

11

Understanding Security

11

Installation

Installing Project for Development

Please refer to the installation guide for detailed information and steps to get the project set up and ready for development on your local machine before proceeding.

Upgrading Node Package Manager (NPM) Dependencies

- To upgrade NPM dependencies, run “npm update” via the terminal in the root directory of where the npm dependencies are installed.
- If using a new NPM dependency and “npm update” does not fully resolve errors in the codebase, run “npm install <name_of_package>” in the root directory of where the npm dependencies are installed
- With the current project setup, you will need to run an npm update in the following locations:
 - root directory of the project
 - frontend directory of the project
 - frontend docker container terminal or restart the frontend docker container

Backend

Expanding the Application Programming Interface (API)

- All API controllers are located in the “server/src/main/java/edu.ncsu.edu.armolecules/controllers” directory of the project
 - When adding a new API controller add within this directory
 - Maintain the following naming convention: [Entity]Controller.java
- Use EnvironmentalFactorController for reference on how to create new API controllers
 - Ensure all necessary spring framework imports are used
 - Maintain consistency with error handling in controllers
 - We have wrapped our API controller methods in Try - catches
 - We have handled all possible error checks (ideally an internal server error should never happen)
 - Changed all returns for the API controller methods to return response entities with response codes:
 - 404 - Not Found
 - 400 - Bad Request
 - 500 - Internal Server Error
 - 200 - Ok

```
@GetMapping(path = "/factors/{id}")
public ResponseEntity<> getEnvironmentalFactor(@PathVariable("id") Integer id) {
    try {
        if (id == null || id < 0) return new ResponseEntity<>(new MessageDTO(message: "ID is not a non-negative integer."), HttpStatus.BAD_REQUEST);
        Optional<EnvironmentalFactor> factor = environmentalFactorRepository.findById(id);
        if (factor.isEmpty()) return new ResponseEntity<>(new MessageDTO(message: "The Environmental Factor does not exist"), HttpStatus.NOT_FOUND);
        EnvironmentalFactor f = factor.get();
        EnvironmentalFactorDTO dto = EnvironmentalFactorDTO.fromEnvironmentalFactor(f);
        String fullPath = "src/main/resources/static";
        String imgPath = fullPath + f.getImagePath();
        Path path = Paths.get(imgPath);
        byte[] fileContent = Files.readAllBytes(path);
        dto.imageData = "data:image/png;base64," + Base64.getEncoder().encodeToString(fileContent);
        return new ResponseEntity<>(dto, HttpStatus.OK);
    } catch (Exception e) {
        return new ResponseEntity<>(new MessageDTO(message: "Unable to get Environmental Factor"), HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

Figure 1: Example GET API method with error handling format with response entity return values

Database

- The Spring framework uses Hibernate as an ORM to configure the database with entities defined by Java classes. Reference the internet for a basic explanation on what ORM's are and what Hibernate is if unfamiliar.
- Reference "Molecule" for an example of how to create a new entity in the database
- Reference "MoleculeRepository" for an example of how to create a repository

1. Define a new entity:

1.1. Add a new Java class in the "server/src/main/java/edu.ncsu.csc.armolecules.entities" package

1.2. Ensure there is the `@Entity` annotation

1.3. Define each "column" of the entities as fields within the class

1.3.1. private fields with public getters and setters

1.3.2. best practice to generate getters and setters using IntelliJ (Ctrl + A: Getters and Setters) so they are formatted for Hibernate correctly.

* Reference the Question entity for examples of how to define foreign key constraints or specify specific restraints on fields.

* Important to include an empty constructor for Hibernate to initialize objects, but it is best practice not to use this constructor within the code of your project.

2. Define a repository for entity defined in 1:

2.1. Add a new Java interface in the "server/src/main/java/edu.ncsu.csc.armolecules.repositories" package that extends the existing Spring CrudRepository

* By default, you do not need to define any methods within this interface, but you can define custom methods with names that match the fields within your entity and Hibernate will automatically wire them correctly on the backend.

NOTE: Sometimes you need to change an entity relationship or structure in the database. It might be possible to create a migration for the entity data, but more than likely it is much easier to simply drop the affected table from the database and have Hibernate regenerate it from scratch. It is ill-advised to make dramatic changes to already existing entities, as this could be hard to create migration scripts for in a production environment.

Frontend

Frontend Folder Structure

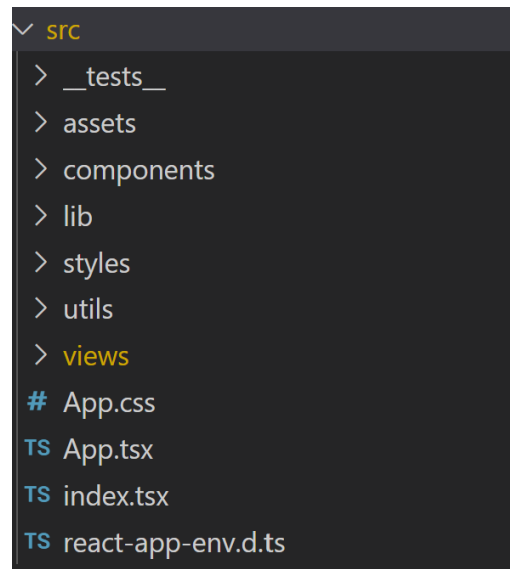


Figure x: Frontend Folder Structure

- We have restructured the entire frontend development folder to follow best practices and for ease of development.
 - Descriptions of what is contained in each folder, naming conventions and other information pertaining to each folder can be referenced in the READMEs in each respective folder in the Github repository.
- App.tsx contains all of the routing for the frontend.
 - Any component added in the frontend must be mapped in this file in the return body in order to view it on the web page.
- AuthProvider is a react context that gives information about the authentication token (i.e. is the user authenticated?) to the rest of the components that it encapsulates.
 - ProtectedRoute is a route that checks the information given to it by AuthProvider so that it can direct the user to the login page to renew their information.
 - The login page on its own handles redirecting back to the page the user was previously logged in on (the previous page information is supplied from the ProtectedRoute).
- To add a new page to the frontend:
 - create a folder under “components”

- create a file under created folder: NewComponents.tsx
 - any respective css or helper files in the “styles” and “utils” folders respectively
- ensure to add a new route in App.tsx for the new component
- poking around the existing frontend files and making edits and changes is the best way to learn and understand how react works.

```
import React from 'react';

function NewComponent() {
  return (
    <div></div>
  );
}

export default NewComponent;
```

Figure y: Example format for new frontend component file

AR Functionality

All of the functionality for the AR view is contained in .js files in the frontend.

When a user navigates to the AR View, control is passed to ARApp.ts, which is the control center for all the AR functionality. To initialize the system, an API call retrieves all the molecules in the system and then passes each one to the MoleculeLibrary which builds a list of molecules. Once the system is initialized, there is a continual loop in the run function in ARApp.ts that calls update and render functions. Update will update the screen and will scan for new markers, and then render will render what was found or remove objects without markers. Each molecule has an update function which will set them as active and therefore able to be viewed on the screen. There is also a function called changeActiveMolecule that is called when a new marker is found. This function sets the new molecule as active. Finally, the factors are updated, meaning if a factor is found then it is compared to the active molecule triggers to see if a state change needs to occur. This mostly works but if there are two or more factors defined then either of them will trigger the state instead of requiring both to be present.

Most of the MoleculeLibrary file could be replaced with functions in the MoleculeHelper file but there was not enough time to make these changes. Additionally, all of the files in lib/js should be converted to helper files and the overlap between the other helper files removed (for example the molecule endpoints are called twice).

The molecules contain all of the objects that are used in rendering the molecules but this is not a good idea because the molecules contain the factors and the tags which might be updated. If the objects are updated the molecules would need to be resaved but this will require a lot of changes to the frontend. Ideally, the system should get a list of all the factors and handle them separately from the molecules (in perhaps a FactorLibrary type setup). Additionally, figuring out a way to not have to store all the molecules in the frontend would speed up the application. Maybe storing a list of markers to molecules/factors that can be referenced to pull the proper molecule or factor via an api call so that only the necessary information is on the frontend.

Testing

*adopted from previous team

Setting up Environment for Testing

1. Install MySQL Workbench on your local machine
 - 1.1. Add a new user to the database
 - 1.1.1. Click on the Server tab and select Users and Privileges
 - 1.1.2. Navigate to the login section
 - 1.1.3. Select Add Account and create new account: ({username}
{password})
 - 1.2. Create an empty schema
 - 1.2.1. Navigate to the local instance of the database
 - 1.2.2. Create an empty schema by right clicking in the schemas panel and name it: molecule_db
 - 1.3. Give the newly created user privileges to the schema created in 2.
 - 1.3.1. Click on the Server tab and select Users and Privileges
 - 1.3.2. Select Schema Privileges: Add Entry
2. Update "application.properties" file on local project
 - 2.1. Change "spring.datasource.url" to equal "jdbc:mysql://localhost:3306/
molecule_db?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=EST"
 - 2.2. Change "spring.datasource.username" to equal the username you setup for MySQL Workbench and "spring.datasource.password" to equal the password you setup for MySQL Workbench
 - 2.3. Change "ar.app.jwtSecret" to equal "arSecretKey"
3. Run tests using gradle
 - 3.1. Install gradle version 6.1.1
 - 3.2. Run command "gradle build" to run tests
 - 3.3. Run command "gradle build jacocoTestReport" to run tests and get coverage reports

Adding to Junit Testing

- New test files should be added in the "server/src/test/java/edu.ncsu.edu.armolecules" directory of the project
- The tests should be added in their corresponding folders under the test directory of the project i.e controller tests in the controllers folder, entities tests in the entities folder, etc...

- Thoroughly testing controllers will inherently test entities and DTOs
- Test file names should be the same as their corresponding main folder names (ex. AccountController.java → AccountControllerTest.java)
- Use EnvironmentalFactorControllerTest.java as a reference for creating controller tests
 - Ensure @Before Method creates and saves a valid account for use during testing
 - POST and PUT API calls which request multipart files will need to use a specific multipart file format

```
@PostMapping(path = "/factors", consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
```

Figure 2: POST mapping consuming a Multipart Form Data Value

```
factor3 = createEnvironmentalFactor(name: "New Factor", imagePath: "NewPath", unit: "Factor Unit",
    discrete: false, defaultValue: 10, minValue: 0, maxValue: 20, description: "New Factor Description");

MockMultipartFile factorFile = new MockMultipartFile(name: "dto", originalFilename: "",
    contentType: "application/json", mapper.writeValueAsBytes(factor3));

mvc.perform(multipart(urlTemplate: "/factors")
    .file(factorFile))
    .andExpect(status().isOk());
```

Figure 3: Example for testing POST request in Figure 1

**** Response Controller Tests:**

- 2 tests are currently failing due to authentication issues
- This is likely due to @WithMockUserRole as a requirement in tests can only have one user role throughout the test.
- With how the test is set up an instructor has access to create the session and a student has access to join the session, but both cannot be performed in the test simultaneously with one user role
- To fix this, the test needs to be reformatted to allow each API call to only use a specified role instead of using the mock user role tag

**** Molecule Controller Tests:**

- Valid test cases are currently failing and have been commented out.
- The issue lies in how the molecule entity is being created and seems to be in regards to the state of the molecule not being recognized.
- We recommend restructuring the creation of a molecule piece by piece to see if the exact issue can be solved.
- Once the format of the molecule works correctly, molecule tests will have over 70% coverage.

****Recommendation:** When developing the application, when any backend changes are made in any component, resolve the test methods surrounding the changes before proceeding with development of other things for ease of resolving test errors.

Security

Understanding Security

*adopted from previous team

- The project uses a standard web development practice of JSON Web Tokens (JWT). With JWT, the user will authenticate with an endpoint using their username and password and receive a token back to be used in any future requests to the backend.

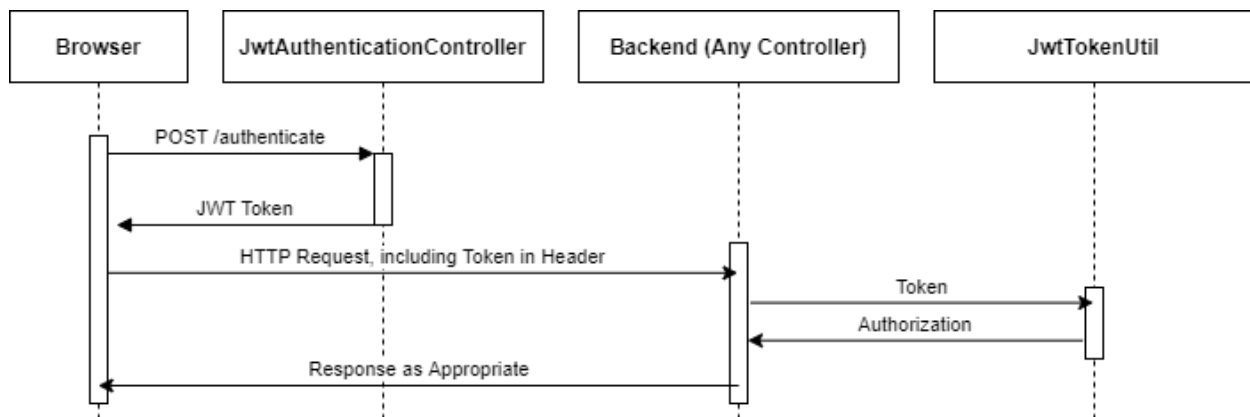


Figure z: The basic flow of functionality from 1.

- In order to keep track of the JWT token sent from the backend, the system stores the token in React context. There is a custom instance of Axios set up to inject our JWT token into each request sent to the backend, and use this instance throughout the entire frontend to make sure that all requests have proper authentication information.
- The backend is able to decipher received JWT tokens to identify the user associated with them. Using the gained user information, the role of the user can be identified and determined if they are authorized to perform the action desired. If they are not, the system will automatically return an error status of 401 Unauthorized.
- The functionality for handling JWT authentication and security in the backend takes place in the following files:
 - controllers/JwtAuthenticationController

- Contains the “/authenticate” endpoint that is hit to log in to the system and get a JWT token.
- security/JWTAuthenticationEntryPoint
 - Handles authorization errors when the incoming request is not authorized to access the specified resource.
- security/JWTRequestFilter
 - Performs JWT token checking on all incoming requests to the system, actually verifying if the user provided a valid JWT token and if they are authorized.
- security/JWTTokenUtil
 - Performs common JWT Token tasks such as generation of tokens and validation of tokens.
- AppWebSecurityConfig
 - Sets up route security to enable JWT authentication checking on all requests.